Lab: Distributional Regression

ACTL3143 & ACTL5111 Deep Learning for Actuaries

Download the freMTPL2sev and freMTPL2freq datasets.

We want to look at severity, but the covariates are stored in the frequency dataset, thus we need to merge the two datasets and drop the `ClaimNb` column from the frequency dataset.

```
import pandas as pd

sev_no_covars = pd.read_parquet('freMTPL2sev.parquet')
freq_df = pd.read_parquet('freMTPL2freq.parquet')

# Just pull out the covariates from the frequency dataset
covariates = freq_df.drop(columns=['ClaimNb'])

# Merge the severity data with the policyholder covariates
severity = pd.merge(sev_no_covars, covariates, on='IDpol', how='left')
severity = severity.dropna()
severity
```

	IDpol	ClaimAmount	Exposure	VehPower	VehAge	DrivAge	BonusMalus	VehBrand	Veh
0	1552	995.20	0.59	11.0	0.0	39.0	56.0	B12	Die
1	1010996	1128.12	0.95	4.0	1.0	49.0	50.0	B12	Reg
2	4024277	1851.11	0.71	4.0	2.0	32.0	106.0	B12	Reg
3	4007252	1204.00	0.78	4.0	1.0	49.0	57.0	B12	Reg
4	4046424	1204.00	0.86	12.0	0.0	37.0	50.0	B12	Die
...	...	...	...	...	...	...	...	...	...
26634	3254353	1200.00	0.07	4.0	13.0	53.0	50.0	B1	Reg
26635	3254353	1800.00	0.07	4.0	13.0	53.0	50.0	B1	Reg
26636	3254353	1000.00	0.07	4.0	13.0	53.0	50.0	B1	Reg
26637	2222064	767.55	0.43	6.0	0.0	67.0	50.0	B2	Die
26638	2254065	1500.00	0.28	7.0	2.0	36.0	60.0	B12	Die

Now we will try to predict claim severity, i.e. `ClaimAmount`, given the rest.

Data dictionary

- `IDpol`: policy number (unique identifier)
- `Area`: area code (categorical, ordinal)
- `BonusMalus`: bonus-malus level between 50 and 230 (with reference level 100)
- `Density`: density of inhabitants per km^2 in the city of the living place of the driver
- `DrivAge`: age of the (most common) driver in years
- `Exposure`: total exposure in yearly units
- `Region`: regions in France (prior to 2016)
- `VehAge`: age of the car in years
- `VehBrand`: car brand (categorical, nominal)
- `VehGas`: diesel or regular fuel car (binary)
- `VehPower`: power of the car (categorical, ordinal)
- `ClaimAmount`: size of the particular claim (**target**)

Source: Nell et al. (2020), Case Study: French Motor Third-Party Liability Claims, SSRN.

GLM

1. Fit a gamma GLM using `statsmodels` with a log link function.
2. Report the average negative log-likelihood loss on the test set.
3. Compute the dispersion parameter using the code from the slides, and compare to `statsmodels`' implementation.

CANN

1. Fit a CANN model.
2. Report the average negative log-likelihood loss on the test set.
3. Recompute the dispersion parameter using the adjusted model. Hint: use this code from the slides to get the means:

```
mus = cann.predict(X_train, verbose=0).flatten()
```

MDN

1. Fit an MDN with 5 Gamma mixture components to predict the claim severity.
2. Change the distributional assumption from Gamma to InverseGamma. Hint: swap the component distribution in the loss function from the slides:

```
mixture = MixtureSameFamily(
    mixture_distribution=Categorical(probs=pis),
    component_distribution=Gamma(concentration=alphas, rate=betas))
```

3. Report the average negative log-likelihood loss on the test set for both models.

Other Metrics

1. Compare the previous models using point-wise metrics such as MAE, MSE, and declare the best model based on these metrics. Which model is the best?
2. Compare the models using distributional metrics such as CRPS, log-likelihood. Which model is the best? Did the order change compared to the point-wise metrics?

Monte Carlo Dropout

1. Construct any style of neural network to predict the claim severity. Then add dropout to some part of the network.
2. Apply Monte Carlo dropout 2000 times and store the test set predictions. Make a histogram of the average test negative log-likelihoods.